Task 04 – Control Implicit Conversions

1. Objective

To understand how C++ performs implicit and explicit type conversions, identify possible data loss, and use safe conversion techniques in the Metro Smart Card System.

2. Concepts and Analysis

Concept	Explanation	Example
Implicit conversion	Automatic conversion between compatible types	double fare = 30;
Narrowing conversion	Conversion that may lose information	int balance = 250.75;
Integer division	Decimal portion is discarded	7 / 2 = 3
Explicit conversion	Programmer specifies the target type	static_cast<double>(7)
Character promotion	Character converts to integer during arithmetic	'C' + 1 = 68
Boolean conversion	Non-zero values convert to true	bool flag = 5;

Signed/unsigned conversion	Signed value converts to unsigned during comparison	-1 < 1u is false

3. Integer Division and Type Conversion

C++ performs integer division when both operands are integers.

```
int a = 7, b = 2;
double x = a / b;                  // Result: 3
double y = static_cast<double>(a) / b; // Result: 3.5
```

Explicit conversion must be applied before division to preserve the decimal portion.

4. Signed and Unsigned Comparison

When a signed integer is compared with an unsigned integer, the signed value may be converted to unsigned.

```
unsigned int u = 1;
int s = -1;


cout << (s < u ? "s < u" : "s >= u");
```

Output: s >= u

This occurs because -1 is converted to a large unsigned value. Compiler warnings can be detected using -Wall -Wextra.

5. Preventing Narrowing Conversions

Brace initialization prevents unintended narrowing conversions.

```
int amount{30};       // Valid
// int amount{3.7};   // Compilation error
```

Using static_cast makes intentional conversions clear and controlled.

6. Alternative Approaches

Approach	**Advantage**	**Limitation**
Implicit conversion	Simple and convenient	May cause unexpected data loss
Explicit conversion	Clear programmer control	Requires additional syntax
Brace initialization	Prevents narrowing	Rejects unsafe conversions
Signed integers	Supports negative values	Can cause signed/unsigned comparison issues

7. Complexity Analysis

- Time complexity: O(1) for individual type conversions and arithmetic operations.
- Space complexity: O(1) additional space for the variables used.
- Explicit conversion does not significantly change memory requirements.

8. Result

The C++ implementation successfully demonstrates implicit conversions, narrowing conversions, integer division, character promotion, Boolean conversion, signed/unsigned comparison, and explicit conversion using static_cast.

Conclusion: Using explicit conversions and brace initialization improves type safety, prevents accidental data loss, and makes the Metro Smart Card System more reliable.